

LLM-Judge Reranking for Code-Clone Search

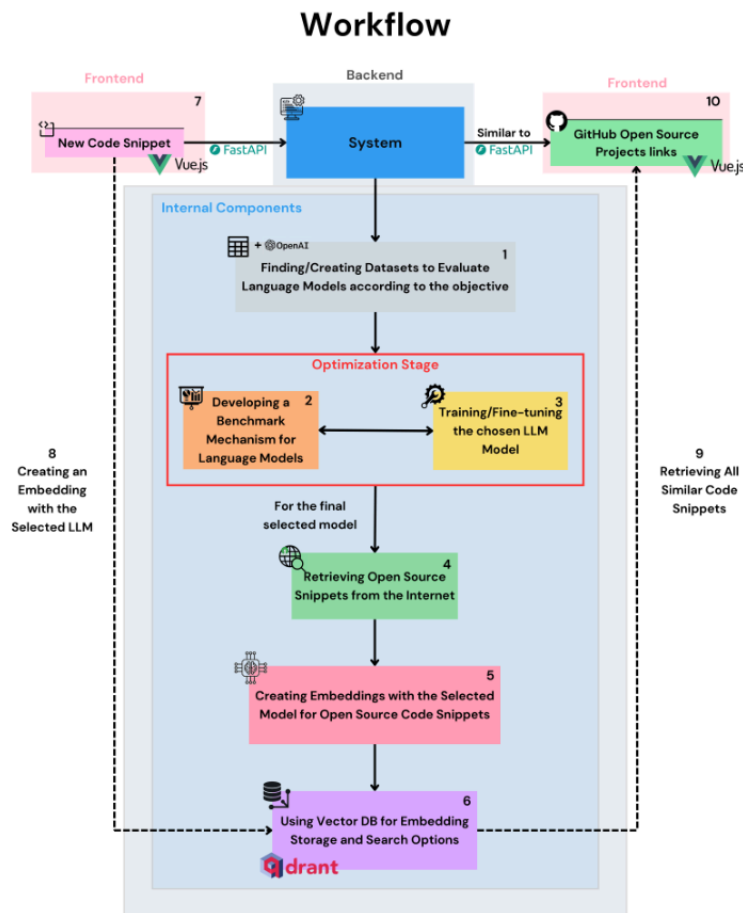
By Yuval Bakirov

PART 1 - BACKGROUND: THE CODEMATCH SYSTEM

Built during my AI Engineer internship at Checkmarx (2024-2025). Included for context - the new work I built with Claude Code is in Part 2.

What CodeMatch does

CodeMatch is a semantic code-search system that helps identify code that may have been copied or derived from open-source projects. It converts code into embeddings, stores them with metadata such as repository, language, and license, and compares a new snippet with the indexed code. This matters because code can keep the same logic even after names, formatting, structure, or programming language change.

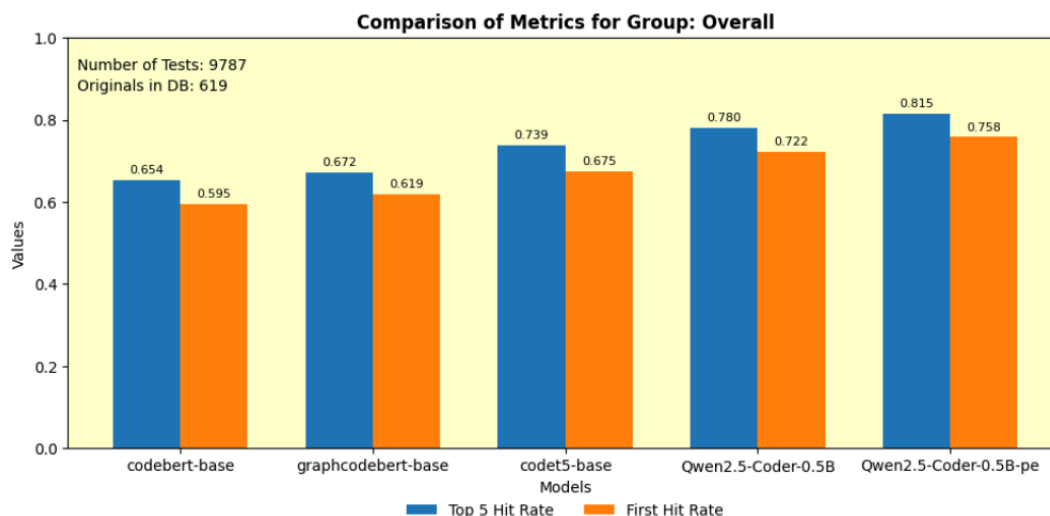


The full system before this project: Vue frontend, FastAPI backend, embedding model, and Qdrant vector database.

How the original system was built

- **Dataset and clone types.** I used four clone types, from near-identical renames (T1) to difficult semantic or cross-language matches (T4). The data combined Rosetta Code with synthetic clones generated using GPT-4o-mini.
- **Benchmark.** I built a modular Python benchmark that compared direct clone matching with top-5 global retrieval and produced results per model and clone type.
- **Model selection.** I compared five embedding configurations. Qwen2.5-Coder with prompt engineering performed best, so it was selected for the retrieval pipeline.
- **Production pipeline.** The final system sourced open-source code from Stack-v2, split it into functions, classes, and modules, filtered and embedded it, removed duplicates, and stored the results in Qdrant. A Vue and FastAPI application exposed the ranked matches and their metadata.

The measured retrieval gap

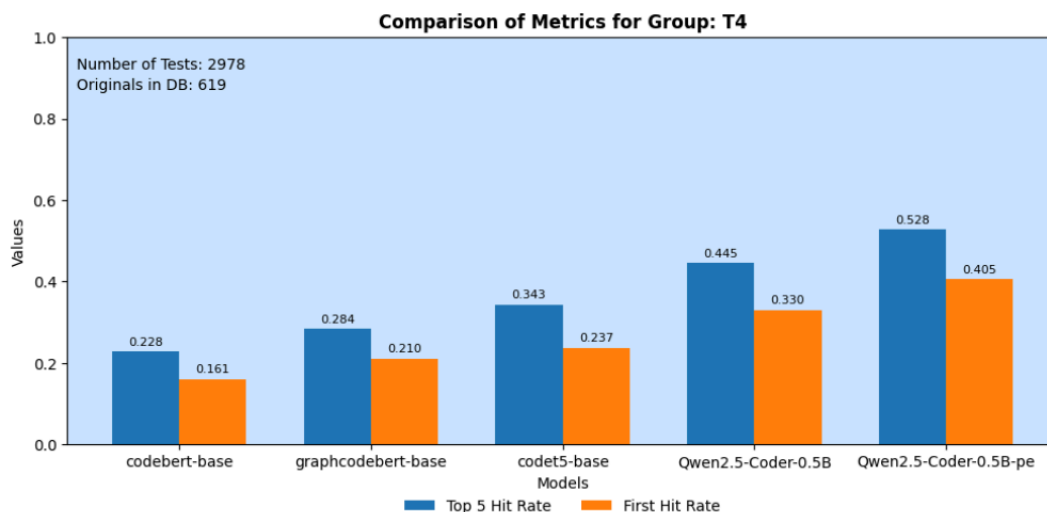


Prior benchmark: five embedding models compared on 9,787 examples, Qwen2.5-Coder-0.5B-pe was selected.

Why embeddings help - and where they fail

Embeddings let the system compare code by meaning instead of exact text. They are fast enough for searching a large collection and can handle renaming, formatting changes, and different languages. They are not perfect: shared imports, boilerplate, tokens, or topic can sometimes make two unrelated snippets look similar.

This problem appeared most clearly on T4 clones, where the same logic is rewritten with a different structure or in another language. Even the best embedding model dropped sharply on this group.



Hardest category (T4), full test set (n=2,978): the measured weakness that motivated Part 2.

PART 2 - AI-ASSISTED ENGINEERING WITH CLAUDE CODE

1. Executive Summary

Problem, goal, and approach

The embedding search was fast, but the benchmark showed that it struggled with difficult semantic clones. I wanted to test one focused idea: keep the existing search, send only its top five results to Claude, and let Claude reorder them. The goal was to improve Hit@1 without losing any result that the embedding model had already found.

- **Architecture.** I kept the embedding search unchanged and added llm_rerank after retrieval. I also kept the Claude client behind a small interface so the logic could be tested without making real API calls.
- **Experiment.** I selected 160 clones, 40 from each clone type, with a fixed seed. I reused the benchmark's existing Hit@1 and Hit@5 definitions.
- **Success rule.** I would integrate the reranker only if Hit@1 improved, Hit@5 stayed unchanged, and any Claude or network failure returned the original ranking.

Where this project fits: reranking added on top of the existing search pipeline



Claude only reorders the top five results already returned by the embedding search. It does not replace retrieval.

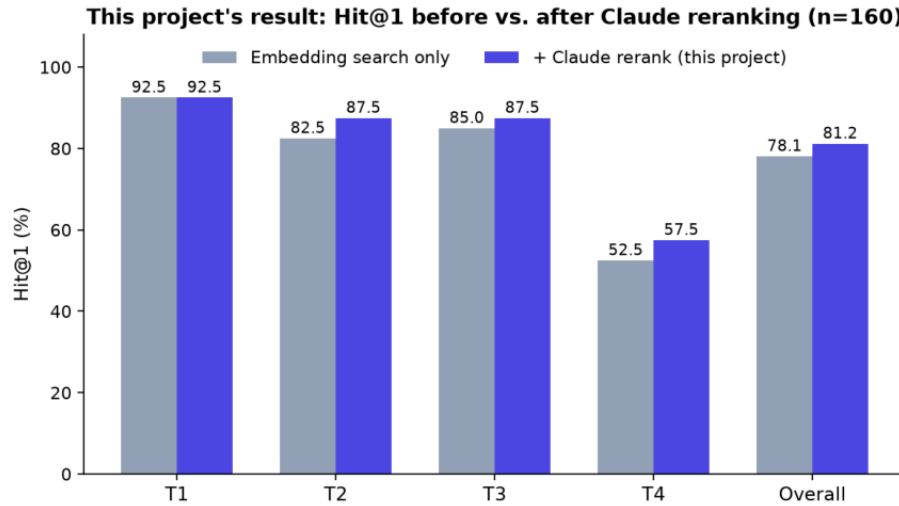
What I decided and what Claude Code helped with

What I decided	How Claude Code helped	How I checked it
Problem, scope, metrics, success criteria, risks, and final decisions	Repository exploration, planning, code changes, tests, debugging, and explanations	Diff review, test output, benchmark results, browser and network checks, and manual inspection

I used Claude Code as a pair-programming tool. It could inspect the repositories, suggest a plan, write code, and help debug. I still chose the problem, the scope, the success criteria, and whether a change was ready to keep.

Outcome

On the 160-clone sample, overall Hit@1 improved from 78.1% to 81.2%, and T4 improved from 52.5% to 57.5%. Hit@5 stayed at 85.6%. That last result was important because it confirmed that the reranker only changed the order of existing candidates, it did not add or remove results.



Measured before/after result on the stratified sample (n=160).

Clone type	n	Hit@1 before	Hit@1 after	Hit@5 before	Hit@5 after
T1 - near-identical	40	92.5%	92.5%	92.5%	92.5%
T2 - renamed	40	82.5%	87.5%	87.5%	87.5%
T3 - restructured	40	85.0%	87.5%	90.0%	90.0%
T4 - semantic / cross-language	40	52.5%	57.5%	72.5%	72.5%
Overall	160	78.1%	81.2%	85.6%	85.6%

How I worked with Claude Code

I did not ask Claude Code to build the whole project in one prompt. I worked through it in small steps:

- 1. Understand the existing system. I first made sure Claude Code had traced the architecture, benchmark, metrics, and T4 weakness.
- 2. Decide what success means. Before coding, I defined the sample, the metrics, the fallback behavior, and the result that would justify integration.
- 3. Ask for a plan before code. I asked Claude Code to show where the reranker should be added, what assumptions it was making, and what could fail.
- 4. Work one change at a time. After each change I reviewed the diff and asked for an explanation whenever I did not fully understand the code.
- 5. Check the result myself. I used tests, the benchmark, the running applications, browser checks, and network output instead of treating Claude's explanation as proof.
- 6. Go back when the evidence is wrong. When something failed, I traced the cause, changed the design, added a regression test, and ran the relevant checks again.

2. Prompts and Development Process

How I gave Claude Code the right context

Before asking for code, I gave Claude Code the real project context: the two repositories, the retrieval and evaluation flow, the current metrics, the measured T4 weakness, and the fallback behavior I wanted. I asked it to inspect the flow and explain its plan before editing anything.

Example of a planning prompt

Before changing code, trace the retrieval and evaluation flow. Show me the smallest place where reranking can be added, the assumptions you are making, and the main failure cases. Keep the embedding pipeline unchanged. Propose a plan first, do not implement yet.

This is a representative prompt, not a word-for-word transcript. Most of my requests followed the same pattern: here is the context, here is the current task, here are the constraints, and here is how to check the result.

What I included in each request

Prompt part	What I gave Claude Code	Why I included it
Context	The relevant files, current flow, benchmark results, and exact weakness	Kept the answer tied to the real codebase
Current task	One concrete change, such as the client interface, rerank logic, or focused tests	Made the diff smaller and easier to review
Constraints	Keep retrieval unchanged, preserve original order on failure, and avoid unrelated edits	Protected the behavior I cared about
How to check it	Run the focused tests, show the diff, explain the choices, and list what is still uncertain	Made Claude Code show evidence instead of only saying it was done

Example of an implementation prompt

Implement only the step we agreed on. Reuse the existing abstractions and keep the original ranking if the judge fails. Add focused tests, run them, show me the diff, and explain anything I would need to defend in a review.

Two different kinds of prompts

- **Prompts to Claude Code.** These were the instructions I used while exploring the repositories, planning, implementing, testing, debugging, and preparing the project case study.
- **The prompt inside the product.** This prompt was sent by the reranker itself. It explained the embedding failure mode, supplied one query and five candidates, and asked for one structured judgment per candidate.

I kept these two layers separate. A good runtime answer does not prove that the surrounding code handles failures correctly. I still needed tests and a real end-to-end run.

Where I changed direction

What happened	What I decided	How Claude Code helped	How I checked it
The benchmark showed a clear T4 weakness	Add a second-stage judge without replacing retrieval	Trace the flow and find the smallest insertion point	Review the architecture and confirm the same top-five candidates remain
8 of 160 JSON responses failed to parse	Do not hide it with retries, change the output interface	Trace the escaping problem and implement forced tool use with a schema	Rerun the same evaluation: 0/160 failures, plus regression tests
The demo crashed with unhashable type: Series	Fix the wrong data assumption, not only the exception	Trace duplicate rows that caused a lookup to return multiple values	Run the regression test and the real application
Offline Hit@1 improved and Hit@5 stayed the same	Only then connect the feature to the live system	Wire the tested module into the backend and UI	Run fallback tests, browser checks, API checks, and the demo
Docker setup was taking too much of the time budget	Time-box it and keep a fallback plan	Run the slow build in the background and report progress	Continue only while it was still supporting the project

How I reviewed Claude Code's work

- I asked questions when I did not understand. Several times I stopped the work and asked for a file-by-file or data-structure explanation before approving the next step. I needed to understand the final design well enough to defend it myself.
- I gave specific feedback. Instead of asking Claude Code to 'make it better,' I explained what was wrong: the demo was unclear, the write-up was too long or too short, or the evidence was hard to find.
- I made it verify claims. When a claim depended on GitHub, test output, the running application, network requests, or current documentation, I asked Claude Code to check the real source instead of answering from memory.
- I made the final scope decisions. Claude Code could suggest an implementation, but I chose the sample size, success criteria, scope boundaries, integration point, and risks I was willing to accept.

Claude Code features I actually used

Feature I used	Where I used it	Why it helped
Repository exploration	Read both repositories and traced the retrieval, evaluation, and integration paths	Stopped me from designing around assumptions
Task tracking	Split the build into nine tasks covering setup, module, tests, evaluation, demo, and documentation	Kept the current step and remaining work clear
Background execution	Ran the 160-call experiment and Docker/application builds in the background	Let me keep reviewing and planning while slow work continued
Browser connector (MCP)	Opened the Streamlit and CodeMatch apps, clicked through them, and checked network and console output	Verified the UI and API behavior directly
Visualization connector	Drew the data flow and experiment pipeline during the session	Made the system easier to inspect and explain
Git and command checks	Used status, diff, log, fetch, test runs, API calls, and service health checks	Gave me evidence instead of a verbal claim
DOCX and PDF skills	Built and rendered the project case study	Helped catch layout and readability problems
Long session context	Kept the work in one session from project selection through implementation and write-up	Preserved decisions while the tasks were still closely connected

I did not try to use every Claude Code feature. I used a feature when it saved time, kept context clear, or gave me stronger evidence.

Features I did not use in this project

Feature	Why I skipped it here	When I would use it
Project CLAUDE.md	This was one short project session, so I supplied the context directly	For an ongoing team repository, I would record the architecture, commands, fallback rules, data boundaries, and conventions
Custom Skill	The evaluation flow was new and still changing	If the team repeated it, I would package sampling, running, validation, comparison, and reporting into one reusable Skill
Hook	I wanted to see and control the test runs while the workflow was still evolving	After the workflow stabilised, I would add a hook for the focused reranker tests or a completion check
Subagent	The architecture, experiment, and integration decisions depended on the same context	I would use a separate reviewer for security, privacy, prompt injection, or an adversarial production review
Additional MCP integrations	The evidence I needed was already local, in Git, or in the running applications	I would connect GitHub or observability only after reviewing authentication, trust, prompt-injection, and data-governance risks

3. Critical Reflection: How I Checked the AI's Work

What I trusted

What I relied on	What it told me	Where I used it
Tests and the benchmark	Whether the deterministic behavior and measured accuracy were correct	Ranking, parsing, fallback behavior, and result claims
Running applications and network output	Whether the integration worked end to end	The Streamlit demo and live CodeMatch path
Claude's explanation and the diff	Why a change was made and what to inspect	A useful starting point, but never proof on its own

How I decided whether to keep a change

- **Keep it.** I kept a change only when it matched the agreed scope, I understood the implementation, the relevant tests passed, the end-to-end behavior worked, and the remaining risk was clear.
- **Revise it.** I asked for another iteration when the basic direction was useful, but the evidence was incomplete, an assumption was still untested, or the diff changed more than I had requested.
- **Reject it.** I rejected output that changed the wrong part of the system, hid a failure with retries, weakened fallback behavior, made a claim without evidence, or could not be reproduced.

My clearest example: invalid JSON

The first full run had 8 parse failures. I did not hide them with retries or just ask Claude to 'return valid JSON.' I changed the interface to forced tool use with a schema, reran the same 160 cases, reached 0 failures, and added tests for the exact problem.

I treated Claude's explanation as a useful clue, not as ground truth. The benchmark decided whether ranking quality improved, tests checked the failure behavior, and the running application showed whether the integration actually worked.

4. Testing Approach

Why I used different types of tests

Test type	What it covered	Why I used it
Unit and contract tests	Prompt building, parsing, unknown IDs, missing fields, sorting, metrics, labels, and CLI configuration	Fast checks for deterministic logic and known edge cases
Tool-use tests	Schema extraction, missing tool blocks, and candidate IDs that were never sent	Protected the structured-output boundary that replaced free-text JSON
Fixture integration test	Load, sample, judge, rerank, and evaluate with a fake Claude client	Checked that the modules worked together without paying for live API calls
Live-integration safeguards	Provider and network failures, fallback order, partial content, and duplicate labels	Protected the user-facing search path when dependencies or data were imperfect
160-clone evaluation	Hit@1 and Hit@5 before and after reranking on the fixed sample	Measured whether the AI feature improved the actual target behavior
Manual and browser checks	The Streamlit demo, live CodeMatch app, a real Claude call, UI state, network requests, and console output	Covered behavior that isolated tests could not prove

Together, these layers covered different questions. The project had 23 reranker and benchmark tests, 3 focused live-integration safeguards, the 160-clone evaluation, and direct checks in the browser and running application. Unit tests alone would not have proven the model quality or the live behavior.

What I did not test

- **Statistical confidence.** The sample was reproducible and balanced across clone types, but it was small compared with the full dataset. I did not calculate confidence intervals or statistical significance, so I treat the improvement as a good signal for a larger evaluation, not a universal claim.
- **Production load.** I did not measure latency, throughput, concurrency, rate limits, cost under load, or long-running provider reliability.
- **Other models and data.** The experiment used one embedding model and the existing Rosetta-Code-based languages and domains. I do not yet know how well the result transfers to other models or real production code.
- **Security and code-data handling.** I documented prompt injection, sensitive-code handling, retention, regional processing, and provider governance as risks, but I did not harden or audit them in this project.
- **Run-to-run variation.** The full runs differed slightly. I did not measure stability across repeated trials, prompt versions, or future Claude model versions.

Problems I found and what I changed

Failure 1 - Claude returned invalid JSON

In the first full experiment, 8 of 160 responses failed to parse because quotation marks inside Claude's explanations broke the hand-written JSON. A few manual examples would probably not have exposed a 5% failure rate. I replaced the free-text response with forced tool use and a defined schema, ran the same experiment again, reached 0/160 failures, and added regression tests.

Failure 2 - A hidden data assumption crashed the demo

The Streamlit demo crashed with `TypeError: unhashable type: 'Series'`. The lookup assumed one row per clone, but the data actually contained five candidate rows per clone, so the lookup returned several values. I fixed the data assumption before grouping and added a regression test instead of patching only the visible exception.

Failure 3 - The live integration needed better fallback handling

When I reviewed the live path, I found that provider errors and incomplete candidate metadata needed stronger handling. I added focused safeguards so provider or network failures return the original embedding order, and cases with missing content or duplicate labels still keep every original result.

What would still be needed for production

The results were strong enough for a guarded integration, but not for an unconditional production rollout. A real launch would still need latency and cost limits, a feature flag, monitoring, privacy approval, adversarial tests, a larger evaluation, and clear rollback conditions.

My rule when working with AI

Claude's answer is a draft. I keep it only when I understand the change, can test the important behavior, and can clearly explain what is still unverified.

5. Demo and Evidence

The recorded demo shows both the standalone Streamlit evaluator and the live CodeMatch integration. Both use the same llm_rerank modules that were covered by the tests, the demo is not a separate mock.

What this shows about how I work

- **I start with a real problem.** The project began with an existing benchmark and a measured weakness.
- **I work in small steps.** I asked for a plan before editing, limited each implementation task, reviewed the result, and changed direction when the evidence did not support the first answer.
- **I ask for proof.** Tests, repeated evaluation, the running application, browser and network checks, and Git history mattered more than an AI-generated explanation.
- **I keep the final decisions with me.** I chose the architecture, scope, success criteria, integration point, and risks that remained open.

The reranker improved the results, but the process was just as important: Claude Code helped me work faster and more safely, while I remained responsible for every key decision.

GitHub – (llm_rerank + benchmark): <https://github.com/YuvalBakirov/codematch-llm-rerank>

GitHub – (CodeMatch system integration): <https://github.com/YuvalBakirov/codematch-system>

Recorded demo video: https://drive.google.com/file/d/1CPDyhbrUXTHNGGnUhLxFCY-KacYnn8AG/view?usp=drive_link

